Enterprise DevSecOps Project

DevSecOps Pipeline Deep Dive

A professional technical walkthrough of CI/CD security gates, artifact flow, GitOps release controls, software supply chain evidence, runtime detection, and observability.

Author	Jagriti Banerjee
Document Type	Pipeline Architecture and Security Deep Dive
Project Domain	Application Security | Container Security | Kubernetes | GitOps | Supply Chain Security

Pipeline Toolchain

Bandit	**pip-audit**	**Trivy**	**Docker**	**GHCR**
ArgoCD	**Cosign**	**Syft**	**Falco**	**Grafana**

Private Lab Deliverable

1. Executive Overview

This document provides a deep technical explanation of the DevSecOps pipeline implemented for the Enterprise DevSecOps project. The pipeline was designed to demonstrate how code, dependencies, containers, Kubernetes manifests, registry artifacts, GitOps deployments and runtime detections can be connected into one evidence-driven security workflow.

The project uses a deliberately vulnerable Flask application as the application security target and surrounds it with modern DevSecOps controls: SAST, software composition analysis, filesystem scanning, container image scanning, SARIF reporting, registry publication, image signing, SBOM generation, GitOps deployment, admission control and runtime observability.

Pipeline goal Turn every code change into a scanned, traceable and reviewable artifact before deployment.	Security posture Evidence-first controls across code, dependencies, image, Kubernetes and runtime layers.	Project output A recruiter-ready pipeline demonstrating practical DevSecOps, AppSec and cloud-native security skills.

Area	Implemented Control	Evidence Produced
Application code	Bandit SAST and manual proof-of-concept testing	SAST reports, vulnerable route evidence, remediation notes
Dependencies	pip-audit SCA against Python requirements	Dependency vulnerability report and package upgrade plan
Container image	Docker build, non-root runtime, Trivy image scan	Image scan report, non-root proof, Docker run evidence
Kubernetes deployment	SecurityContext, RBAC, NetworkPolicy, Gatekeeper	Manifest scans, policy blocks, RBAC can-i proof
Supply chain	GHCR, Cosign signatures, Syft SBOM, provenance	Signature verification, SBOM, attestation proof
Runtime security	Falco, Falcosidekick, Prometheus and Grafana	Runtime alerts, metrics queries, dashboard screenshots

2. Document Map and Pipeline Scope

Pipeline Scope

The scope of this deep dive is the project pipeline and the security value created at each stage. It does not claim production-grade SLSA compliance, production secret management maturity, or public target testing. All activity was performed in a controlled private lab.

• In scope: GitHub Actions workflow, scanners, report artifacts, container image build, GHCR publication, GitOps deployment, Kubernetes manifests, supply chain evidence, runtime security telemetry.

• Out of scope: exploitation of external targets, production infrastructure, real user data, public cloud deployment and organization-wide policy enforcement.

• Assumption: the vulnerable Flask app exists intentionally to validate detection, remediation and retest workflows.

3. Pipeline Architecture

The pipeline follows a security-gate model. Independent checks run early to provide fast feedback, then container build and image scanning occur only after code and dependency checks complete. After the image is published, GitOps, supply chain and runtime monitoring extend the pipeline beyond traditional CI into continuous delivery and operational security.

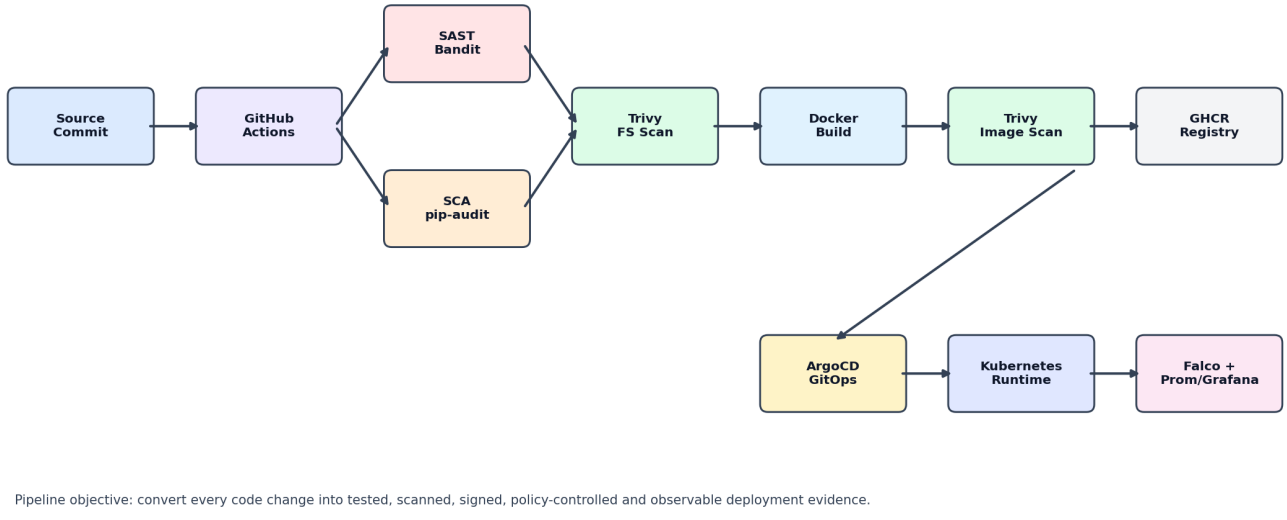


Figure 1. End-to-end Enterprise DevSecOps pipeline flow.

The workflow is intentionally layered. This prevents the project from becoming a simple tool list: each stage produces an artifact that supports the next control. For example, source code enters SAST and SCA, the Docker image enters Trivy and GHCR, the registry artifact is signed with Cosign, and the Kubernetes deployment is managed through ArgoCD and protected by policy controls.

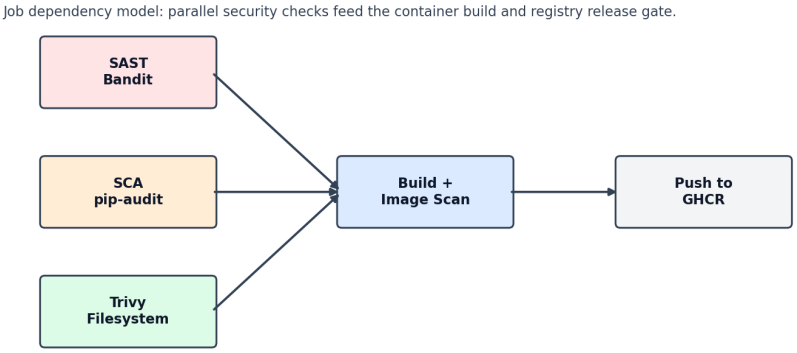


Figure 2. GitHub Actions job dependency model.

4. Source Control and Trigger Strategy

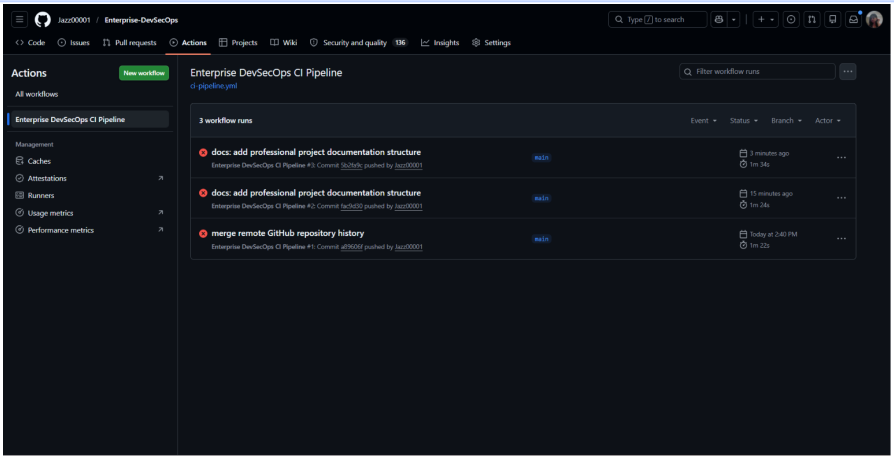
The pipeline is triggered on changes to main, develop and pull requests into main. This supports both continuous validation and review-based security controls. Pull requests validate code before merge, while main branch pushes publish the container artifact.

```
on:
push:
branches: [main, develop]
pull_request:
branches: [main]

permissions:
contents: read
security-events: write
packages: write
actions: read
```

Trigger or Permission	Pipeline Meaning	Security Value
push to main/develop	Runs CI checks on active branches	Prevents unvalidated changes from silently entering the repo
pull_request to main	Runs checks before merge	Creates a review gate before protected branch integration
security-events: write	Allows SARIF upload to GitHub code scanning	Centralizes scanner findings into the platform security view
packages: write	Allows GHCR image publication	Restricts registry push to authorized workflow context

The design separates evidence generation from release publication. Security checks run on both branch and pull request workflows, but image publishing is limited to the main branch path to reduce accidental release artifacts from development branches.



Evidence: GitHub Actions pipeline execution view showing the CI workflow running across security stages.

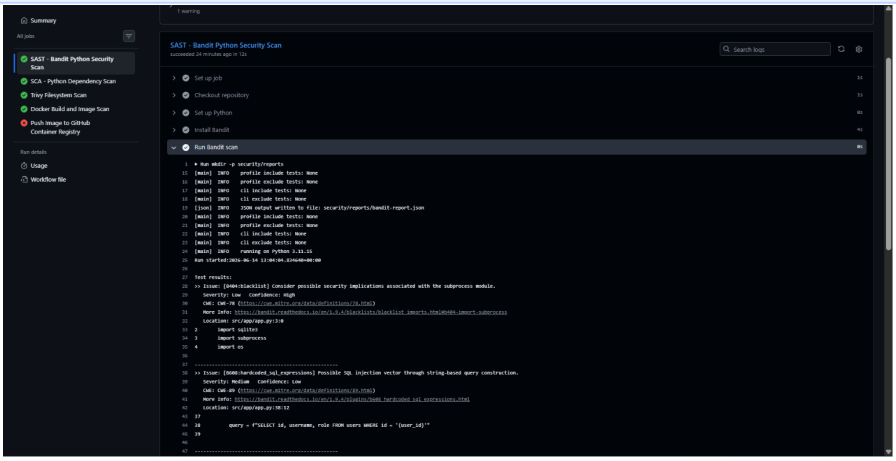
5. SAST Stage - Bandit Python Security Scan

The SAST stage analyzes Python source code before packaging. In this project, Bandit was used to identify risky coding patterns such as shell execution, subprocess usage, unsafe rendering and injection-prone behavior. The stage is positioned early because source-level issues are cheaper to fix before image build and deployment.

```
bandit -r src/app -f json -o security/reports/bandit-report.json || true
bandit -r src/app || true
```

SAST Focus	Project Example	Pipeline Output
Command execution	subprocess.run with shell=True in /ping route	Bandit finding and command injection report
Unsafe rendering	render_template_string with user-controlled input in /hello route	Unsafe rendering finding and remediation guidance
Insecure logic patterns	Manually reviewed vulnerable Flask routes	Security findings, AppSec test cases and retest criteria

In this lab, Bandit is intentionally configured in evidence mode using || true. This means the workflow captures findings without stopping the training pipeline. A production version should move high-confidence high-severity issues to fail-closed mode after baseline triage.



Evidence: Bandit job output captured in GitHub Actions.

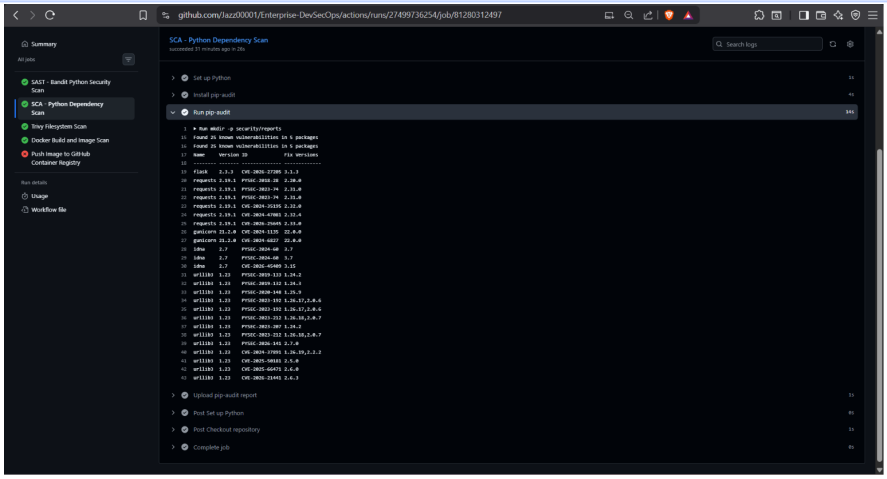
6. SCA Stage - pip-audit Dependency Vulnerability Scan

The software composition analysis stage checks Python dependencies declared in requirements.txt. This is essential because a secure application can still inherit risk from vulnerable third-party packages. The lab intentionally includes an older requests package to demonstrate dependency vulnerability detection.

```
pip-audit -r src/app/requirements.txt -f json -o security/reports/pip-audit-report.json || true
pip-audit -r src/app/requirements.txt || true
```

Dependency	Observed Purpose	Security Review Focus
Flask==2.3.3	Web application framework	Framework patch level and transitive dependencies
requests==2.19.1	HTTP client library	Known vulnerable dependency example for SCA evidence
gunicorn==21.2.0	Production WSGI runtime	Runtime server patching and image-level exposure

Dependency scanning strengthens the pipeline by adding a second lens that SAST does not cover. Bandit inspects custom code. pip-audit inspects published packages and known advisories. Both are needed to represent a realistic AppSec pipeline.



Evidence: pip-audit job output in the CI workflow.

7. Trivy Filesystem and Misconfiguration Scanning

The filesystem scan stage reviews repository content for vulnerabilities, secrets and misconfigurations. This expands coverage beyond Python-only checks and helps detect security issues in Dockerfiles, Kubernetes manifests and other project artifacts.

```
uses: aquasecurity/trivy-action@master
with:
scan-type: fs
scan-ref: .
format: table
output: security/reports/trivy-fs-report.txt
severity: CRITICAL,HIGH,MEDIUM
exit-code: "0"
```

Scan Area	Pipeline Purpose	Example Evidence
Repository filesystem	Find vulnerabilities and weak configuration across project files	Trivy filesystem report
Dockerfile	Identify container build and runtime hardening issues	Trivy config and image scan reports
Kubernetes manifests	Identify missing securityContext, resource limits and insecure settings	Trivy k8s manifest scan
Secret exposure	Detect accidental token/key patterns if present	Filesystem and secret scan output

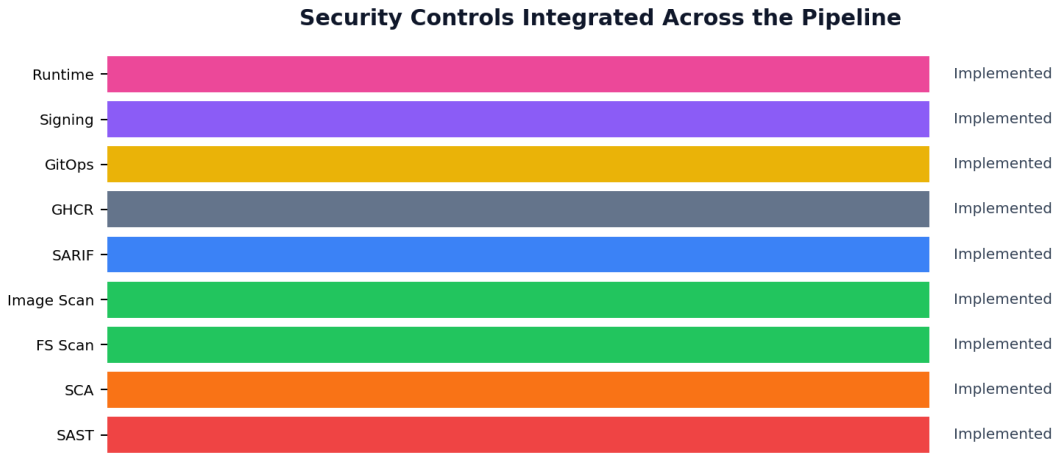


Figure 3. Security controls distributed across pipeline stages.

8. Docker Build and Container Image Scan

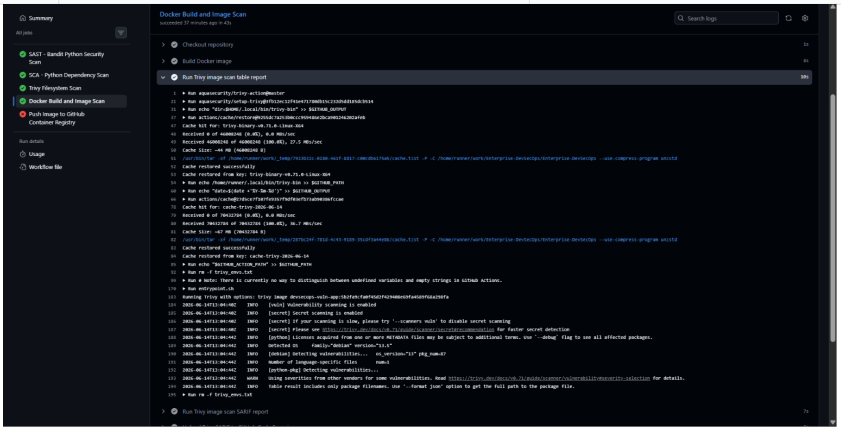
After the early source and dependency checks complete, the pipeline builds a Docker image and scans the resulting image. This stage validates the packaged runtime artifact rather than only the source repository. The image includes a multi-stage Dockerfile, non-root user, Gunicorn runtime and health check.

```
needs: [sast, dependency-scan, filesystem-scan]

- name: Build Docker image
run: docker build -t devsecops-vuln-app:${{ github.sha }} .

- name: Run Trivy image scan table report
uses: aquasecurity/trivy-action@master
with:
image-ref: devsecops-vuln-app:${{ github.sha }}
format: table
output: security/reports/trivy-image-report.txt
```

Build Control	Implementation	Security Value
Multi-stage Dockerfile	Separate builder and runtime images	Reduces final image footprint and unnecessary tooling
Non-root user	USER 10001:10001	Reduces impact of container compromise
Healthcheck	/health endpoint validated by Docker/Kubernetes	Improves operational reliability and deployment readiness
Image scanning	Trivy image scan in CI	Detects OS and package vulnerabilities in the final artifact



Evidence: Trivy image scan executed against the CI-built Docker image.

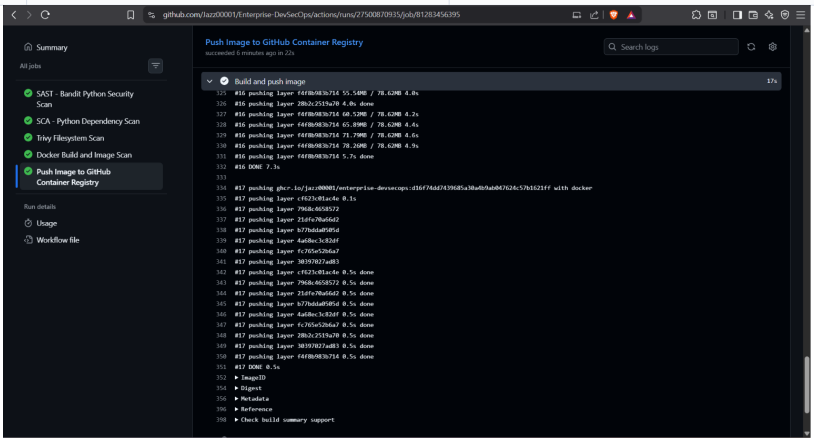
9. Artifact Publishing to GitHub Container Registry

The push-to-registry stage publishes the image to GHCR only from the main branch. This creates a controlled release boundary between CI validation and registry publication. The image is tagged with both latest and the commit SHA to support traceability.

```
if: github.ref == 'refs/heads/main'

tags: |
ghcr.io/${{ env.IMAGE_NAME }}:latest
ghcr.io/${{ env.IMAGE_NAME }}:${{ github.sha }}
```

Registry Control	Why It Matters	Deep Dive Notes
Commit SHA tag	Creates immutable-ish traceability to source revision	Used for incident response and retest comparison
latest tag	Provides simple human-readable deployment pointer	Should be supplemented by digest pinning in production
GITHUB_TOKEN login	Avoids hard-coded registry credentials	Token inherits workflow permissions and repository context
Main branch condition	Prevents accidental release from feature branches	Supports separation between validation and release



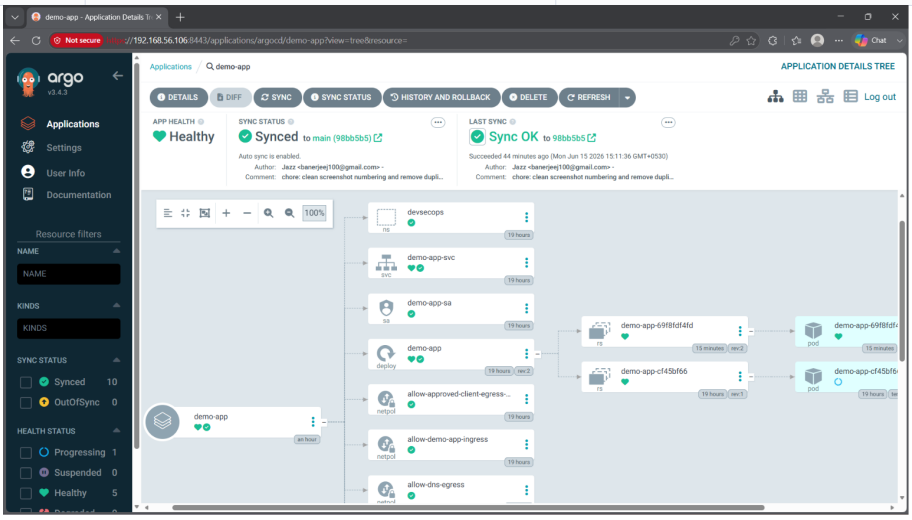
Evidence: GHCR image publication stage completed.

10. GitOps Release with ArgoCD

ArgoCD extends the pipeline from CI into continuous delivery. Instead of manually applying manifests, the Kubernetes cluster is synchronized from the Git repository. This creates a desired-state model where Git is the source of truth and live cluster drift can be corrected.

```
syncPolicy:
automated:
prune: true
selfHeal: true
syncOptions:
- CreateNamespace=true
```

GitOps Feature	Pipeline Function	Security Value
Automated sync	Deploys Git-defined manifests to cluster	Reduces manual drift and undocumented changes
Self-heal	Reverts live state back to Git	Detects and corrects unauthorized configuration drift
Prune	Removes resources deleted from Git	Prevents orphaned resources from lingering
Application resource tree	Shows deployed objects and status	Provides release visibility and audit evidence



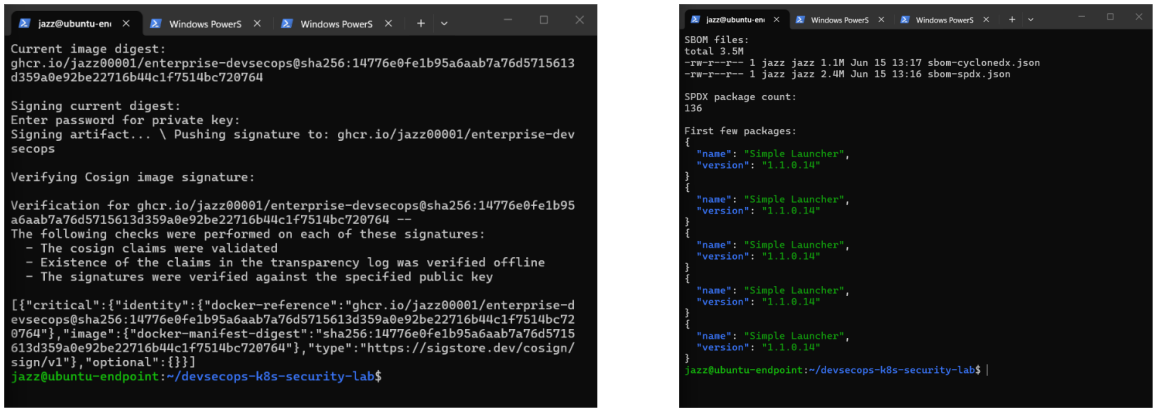
Evidence: ArgoCD resource tree showing Kubernetes resources managed through GitOps.

11. Supply Chain Controls: Cosign, Syft and Provenance

The supply chain phase adds cryptographic and inventory evidence to the image artifact. Cosign is used for image signing and attestation verification, while Syft generates a Software Bill of Materials. This makes the registry artifact more than a container image: it becomes an auditable software supply chain object.

Control	Artifact Produced	Security Purpose
Cosign image signing	Signature associated with image reference	Proves image integrity against the public key
Cosign verification	Verification report	Validates that the image matches the expected signature
Syft SBOM	SPDX and CycloneDX SBOM files	Documents packages and components included in the image
SBOM attestation	Signed metadata linked to image	Binds inventory evidence to the registry artifact
SLSA-style provenance	Build provenance predicate	Captures source, build and image traceability for lab evidence

The lab provenance is SLSA-style evidence generated in a private VM environment. It is suitable for learning, documentation and portfolio demonstration. A production-grade implementation should use a trusted builder with generated provenance and stronger key management.

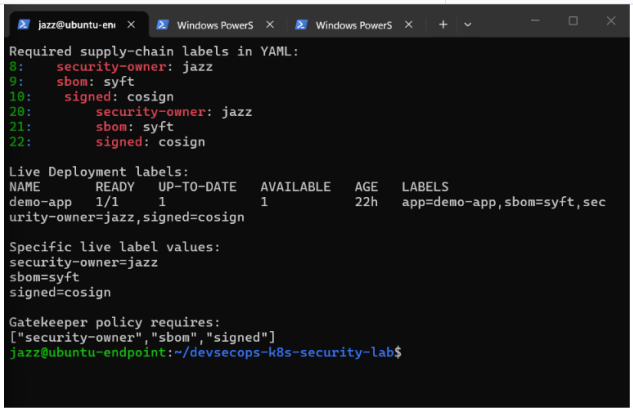


Evidence: Cosign signature verification and Syft SBOM generation.

12. Kubernetes Deployment Security Gates

The pipeline does not stop at building and scanning images. It also applies Kubernetes-level controls that determine whether workloads can safely run. These controls are expressed as manifests, policies and validation tests.

Kubernetes Control	Implementation	Pipeline Security Outcome
Namespace isolation	devsecops namespace with restricted labels	Separates workload and applies baseline pod restrictions
Dedicated ServiceAccount	demo-app-sa with token automount disabled	Limits credential exposure inside the pod
Least-privilege RBAC	Can read ConfigMaps but cannot read Secrets or list nodes	Prevents unnecessary cluster access
Hardened securityContext	runAsNonRoot, UID 10001, no privilege escalation, drop ALL capabilities	Reduces container breakout and escalation risk
NetworkPolicy	Default deny plus allowed ingress/egress	Enforces zero-trust pod communication
Gatekeeper admission	Required supply chain labels on deployments	Blocks non-compliant workloads before admission



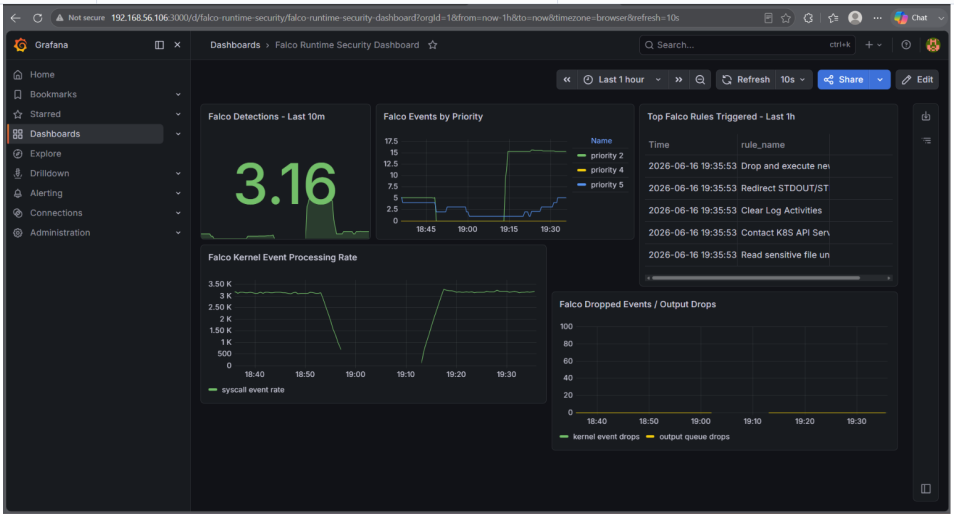
Evidence: Deployment labels supporting supply chain admission policy.

A mature DevSecOps pipeline treats Kubernetes manifests as security-critical code. In this project, manifests are scanned, applied through GitOps and validated with runtime proof such as RBAC denial, NetworkPolicy block results and read-only filesystem checks.

13. Runtime Detection and Observability

Runtime security closes the loop after deployment. Falco detects suspicious container behavior, Falcosidekick forwards events, Prometheus stores security metrics and Grafana visualizes detections. This creates operational feedback that complements pre-deployment security gates.

Runtime Layer	Tooling	Evidence
Threat detection	Falco rules for sensitive file access and privileged behavior	Falco alert logs and runtime screenshots
Event forwarding	Falcosidekick and ServiceMonitor	Prometheus target and metrics endpoint evidence
Metrics storage	Prometheus	Falco metric queries and alert rules
Dashboarding	Grafana	Custom Falco runtime security dashboard



Evidence: Custom Grafana security dashboard for Falco runtime events.

The pipeline therefore becomes continuous: code is scanned before deployment, images are scanned and signed before release, Kubernetes policies enforce admission, and runtime telemetry validates behavior after deployment.

14. Pipeline Control Matrix

Stage	Primary Tool	Input	Output	Gate Type
Source trigger	GitHub Actions	Push or pull request	Workflow execution	Entry gate
SAST	Bandit	src/app Python code	bandit-report.json	Code risk detection
SCA	pip-audit	requirements.txt	pip-audit-report.json	Dependency risk detection
Repository scan	Trivy FS	Repo files	trivy-fs-report.txt	Repo-level security scan
Image build	Docker	Dockerfile + app	Container image	Build gate
Image scan	Trivy image	Built image	trivy-image-report.txt + SARIF	Artifact risk detection
Registry release	GHCR	Scanned image	Tagged registry image	Release gate
GitOps deploy	ArgoCD	k8s/base manifests	Synced cluster state	Delivery gate
Admission control	Gatekeeper	Kubernetes objects	Allow/deny decisions	Policy gate
Supply chain	Cosign/Syft	Registry image	Signature, SBOM, attestations	Integrity and inventory gate
Runtime detection	Falco	Container events	Runtime alerts	Operational gate
Monitoring	Prometheus/Grafana	Falco and cluster metrics	Dashboards and alerts	Visibility gate

This matrix demonstrates breadth and ordering. Early gates catch source and dependency problems; middle gates validate the built artifact; late gates verify Kubernetes policy compliance, supply chain trust and runtime behavior.

15. Failure Handling, Evidence and Retest Strategy

A professional DevSecOps pipeline must define how findings are handled. The lab uses an evidence-first approach: each scanner output is saved, screenshots are indexed, and remediation is verified through retest commands.

Finding Source	Failure Handling	Retest Evidence
Bandit	Review severity, confirm exploitability, patch unsafe code, rerun scan	Bandit output shows no high-risk issue or accepted exception
pip-audit	Upgrade vulnerable dependency, pin safe version, rerun SCA	pip-audit report confirms advisories are resolved
Trivy	Patch base image/package or document exception	New image/config scan report
Gatekeeper	Add required metadata or fix manifest	Bad object blocked, good object allowed
RBAC	Remove cluster-admin binding, replace with namespace RoleBinding	kubectl auth can-i denies secrets and clusterrolebindings
NetworkPolicy	Default deny and explicit allowed paths	Blocked pod times out; trusted pod succeeds
ArgoCD	Drift restored from Git state	Replica drift self-healed and Git changes synced
Falco	Generate safe test event and validate detection	Falco logs and Prometheus query show rule match

Recommended Gate Maturity Path

• Phase 1 - Evidence mode: collect findings without breaking the pipeline while baseline is being created.

• Phase 2 - Advisory gate: fail only on newly introduced critical issues or confirmed exploitable patterns.

• Phase 3 - Enforcing gate: fail on high and critical findings after documented exception handling exists.

• Phase 4 - Continuous control: combine CI gating, GitOps drift correction, admission control and runtime detection.

16. Recommendations and Conclusion

The implemented pipeline is strong for a private lab and portfolio project because it demonstrates the complete DevSecOps lifecycle rather than isolated tool usage. The next improvement is to make the pipeline more production-like by strengthening enforcement, secret handling, artifact immutability and environment separation.

Recommendation	Why It Matters	Priority
Move selected SAST/SCA/Trivy checks from evidence mode to fail-closed mode	Prevents known high-risk issues from passing silently after baseline	High
Pin GitHub Actions and container images by version or digest	Reduces supply chain uncertainty in CI dependencies	High
Use digest-pinned Kubernetes image references	Prevents latest tag drift between scan and deployment	High
Adopt trusted provenance generation in CI	Improves SLSA maturity and reduces manual provenance trust gaps	Medium
Add environment promotion workflow	Separates dev, staging and production release controls	Medium
Add centralized exception tracking	Improves auditability of accepted risks	Medium
Add notification integration	Routes pipeline and runtime findings to Slack/Jira or equivalent	Medium

Overall conclusion

This DevSecOps pipeline demonstrates a realistic security engineering workflow: source code is reviewed, dependencies are audited, the container image is built and scanned, artifacts are published and signed, Kubernetes deployment is controlled by GitOps, admission policies restrict unsafe workloads, and runtime activity is monitored. The project provides strong evidence of practical DevSecOps, AppSec, Kubernetes security and supply chain security capability.

Document prepared by Jagriti Banerjee for the Enterprise DevSecOps project.